Revised Report on Möbius

R0RM — Revised0 Report on Möbius

Draft 4 — February 2026

0. Executive Summary

- Möbius is a Lisp inspired from Scheme. It is the computational core of the Möbius infrastructure: content-addressed, multilingual, authorship-preserving.
- One observation: computation is tree transformation. One data constructor: `cons`. One mechanism: `gamma`.
- Eight value categories: atoms (integer, float, character, string), pairs, empty list, booleans, capsules, combiners, boxes, continuations.
- Three surface syntaxes — round (S-expressions, prefix), curly (braces, infix), spacy (indentation, infix) — all producing identical content-addressed trees.
- Pattern syntax is universal across all three surfaces: `,x` binds, `,(x)` recurses, `(? pred ,x)` guards. No bare identifiers in patterns.
- Top-level definitions are immutable and content-addressed. Boxes (mutable indirection) exist only at runtime inside nested scope.
- Types are predicates. No type declarations. No inheritance. Classification is external and open.
- No quote, no quasiquote. All data construction through `cons` and the `list` library function. All surfaces produce identical hashes. Anonymous combiners (inline `gamma` or `lambda` as arguments) are forbidden in all surfaces — every combiner must be named via `define`.
- **Foundations** (~34 names) are forms and combiners that require the evaluator or runtime — they cannot be written in Möbius. Some have equivalent expansions in terms of others; these are semantic facts the compiler may exploit, not a hierarchy.
- **Base library** combiners are Möbius programs shipped with the system — they have content hashes and live in the store. Any programmer could write them.
- Economy (Hsu, 2019): the ratio of domain-specific names to total names measures how much knowledge scales to a broad set of problems. Möbius minimizes the denominator.
- The "0" in R0RM: this is the revision before revision, the seed before the tree.

1. Content Model

Möbius separates **content** (immutable, content-addressed) from **naming** (mutable, versioned).

1.1 The content store

The content store is an immutable mapping from hashes to trees.

```
store : Hash → Tree
```

Every distinct tree has a unique hash, computed from its structure — atoms, pairs, and references to other hashes. Once stored, a tree cannot be changed. Its hash is its identity.

1.2 The registry

The registry is a mutable, versioned mapping from names to hashes.

```
registry : Name → Hash
```

Names are strings in a natural language. The same hash may have different names in different languages:

```
"odd?"   → 0x7a3f...
"impair?" → 0x7a3f...
"فردي؟"   → 0x7a3f...
```

The registry is the **image** — the living, evolving view into the content store. Updating a name to point to a new hash does not change the old tree; it creates a new version of the registry.

1.3 Registration

Registration converts surface syntax into content-addressed form:

1. **Parse** surface syntax into an AST with names.
2. **Resolve** each name against the current registry, obtaining a hash.
3. **Replace** names with hashes, producing a tree of atoms, pairs, and hash references.
4. **Compute** the hash of the resulting tree.
5. **Store** the tree in the content store (if not already present).
6. **Bind** the name to the hash in the registry.

After registration, the original names are gone from the content. Only hashes remain.

The registrar detects mutually recursive top-level definitions and bundles them into a single content-addressed unit. The dependency DAG is between these units, not between individual definitions (§7.3).

Note: the reader and the registration pipeline may internally represent names as symbols. Symbols are an implementation detail of the toolchain, not a user-visible value type. The constraint "no symbols as values" applies to the content store and to runtime, not to intermediate representations.

1.4 Combiner structure

A combiner in the content store is a tree containing:

- **Atoms**: integers, floats, characters, strings, 128-bit type identifiers — self-hashing.
- **Pairs**: structure — hash is computed from the hashes of car and cdr.
- **Hash references**: pointers to other content (foundations, constants, other combiners).
- **Bound variables**: positions introduced by patterns and used within the same combiner.

Bound variables are local to a combiner and represented by position (de Bruijn indices or equivalent), not by name. This ensures that alpha-equivalent combiners hash to the same value.

1.5 Foundations

Foundations are forms and combiners built into the runtime with reserved hashes — integers smaller than 2^{12}, known to all implementations. Foundations are not stored in the content store — they are intrinsic.

2. Values

All values in Möbius are trees. There are eight categories:

Atoms. An integer, a float, a character, or a string.

Pairs. Two trees joined by `cons`. The only compositional data constructor. All structures — lists, records, tables, matrices — are built from pairs.

The empty list. Written `#nil` in all surfaces. `(eq? #nil #nil)` is `#true`.

Booleans. `#true` and `#false`. Only `#false` is false — it is the sole value that causes `if` to take the else branch. Every other value, including `0`, `#nil`, and the empty string, is considered true.

The void value. `#void` is a pre-defined singleton. It is the conventional return value of side-effecting operations like `box!` and `display`. `#void` is true (it is not `#false`).

Capsules. An opaque value tagged with a 128-bit type identifier (§9). Two trees with identical structure but wrapped in different capsule types are distinct.

Combiners. The result of evaluating a `gamma` or `lambda` expression. A combiner is a tree that, when applied, receives a tree and produces a tree.

Boxes. A mutable indirection cell created by `box`, read by `unbox`, mutated by `box!` (§10). Boxes are the only mutable values in Möbius. They are forbidden in top-level content-addressed definitions.

Continuations. A first-class value representing a point of execution, created by `call/cc` (§12). Continuations are runtime-only — they cannot be stored in the content

store. They are not combiners: you cannot apply them with the application rule. Use `continuation-apply` to deliver a value to a continuation.

There is no distinction between "data" and "code" at the structural level. Both are trees.

There are no symbols as values. Names in source syntax are resolved to content hashes at registration time.

There is no `quote` and no `quasiquote`. All data construction is explicit, through `cons` and the `list` library function. This eliminates the question of what symbols become when quoted — they don't exist as values, and there is no mechanism that pretends otherwise.

3. Three Surfaces

Möbius has three surface syntaxes. All three produce identical content-addressed trees. The surface is a view; the content is the truth.

3.1 Surface declaration

A Möbius source file uses the extension `.mobius` and begins with a surface declaration:

```
#lang round
#lang curly
#lang spacy
```

If absent, behavior is implementation-defined. The surface can also be selected via command-line flag: `--surface=round`, `--surface=curly`, `--surface=spacy`.

3.2 Round surface

Round is the S-expression surface. Prefix notation. Parentheses delimit everything.

```
#lang round

;; constants
(define pi 3.14)

;; combiners with lambda
(define double (lambda (x) (* x 2)))

;; combiners with gamma
(define sum (gamma ((,head . ,(tail)) (+ head tail))
                   (#nil               0)))

;; application
(sum (list 1 2 3))

;; conditional
```

```
(if (> x 0) (+ x 1) (- x 1))

;; sequencing
(begin (display "hello") (display "world") 42)

;; list construction
(list 1 2 3)            ;; => (1 . (2 . (3 . #nil)))
(cons 1 (cons 2 #nil))  ;; => (1 . (2 . #nil))
```

Comments: `;` for line comments, `#;` for datum comments.

No anonymous combiners as arguments. Every `gamma` or `lambda` must be bound to a name via `define`. This ensures surface equivalence — any program written in round can be identically expressed in curly and spacy.

```
;; WRONG — anonymous combiner as argument
(map (lambda (x) (* x 2)) my-list)

;; CORRECT — named combiner
(define double (lambda (x) (* x 2)))
(map double my-list)
```

3.3 Curly surface

Curly uses braces and semicolons. Infix arithmetic with mandatory full parenthesization.

```
#lang curly

;; constants
define pi 3.14;

;; combiners with lambda
define double lambda (x) { (x * 2) };

;; combiners with gamma
define sum gamma {
    case (,head . ,(tail)): (head + tail);
    case #nil: 0;
};

;; application — space-separated arguments, no commas
sum(list(1 2 3));

;; conditional
if (x > 0) { (x + 1) } else { (x - 1) };

;; sequencing — braces + semicolons
```

```
{
    display("hello");
    display("world");
    42
};

;; multi-expression case bodies
define foo gamma {
    case (,x ,y): {
        define z (x + y);
        define w (z * 2);
        w
    };
    case #nil: 0;
};
```

Arguments are space-separated. Comma is reserved for pattern syntax (`,x`, `,(x)`). The expression `f(3 4)` passes the tree `(3 . (4 . #nil))` to `f`.

Infix rule: `(1 + 2)` is valid. `1 + 2 * 3` without parentheses is a **reader error**. The programmer must write `(1 + (2 * 3))` or `((1 + 2) * 3)` explicitly. There is no precedence. The reader translates `(a + b)` to the same tree as round's `(+ a b)`.

Semicolons separate statements. The last expression in a brace block is the return value.

No anonymous combiners as arguments. Same rule as round — every `gamma` or `lambda` must be named.

3.4 Spacy surface

Spacy uses indentation and colons. Infix arithmetic with mandatory full parenthesization.

```
#lang spacy

;; constants
define pi: 3.14

;; combiners with lambda
define double: lambda (x): (x * 2)

;; combiners with gamma
define sum: gamma:
    case (,head . ,(tail)): (head + tail)
    case #nil: 0

;; application — space-separated arguments
sum(list(1 2 3))
```

```
;; conditional
if (x > 0):
    (x + 1)
else:
    (x - 1)

;; sequencing — newlines under same indentation
define foo: gamma:
    case (,x ,y):
        define z: (x + y)
        define w: (z * 2)
        w
    case #nil: 0
```

The colon is spacy's "here comes the body" marker. It appears after `define name`, after `lambda (params)`, after `gamma`, after `case pattern`, after `if (cond)`, and after `else`.

After a colon in inline position, exactly **one expression** follows as the body. For multi-expression bodies, use a newline and indented block. The last expression in a block is the return value.

Semicolons are permitted as inline statement separators (as in Python) but are discouraged.

No anonymous combiners as arguments. Same rule as round and curly. This is the constraint that motivated the universal ban — spacy's indentation-based scoping cannot delimit inline anonymous combiners without ambiguity, so all surfaces share the restriction to maintain equivalence.

The infix rule is the same as curly: mandatory full parenthesization, no precedence, reader translates to prefix trees.

3.5 Identifier lexing

The same lexer rule applies to all three surfaces:

- An identifier starts with an alphabetical character or one of `+ - * / < > = ? ! _`.
- It continues until **whitespace** or a **structural delimiter**: `(`, `)`, `{`, `}`, `;`, `:`, `'`, `` ` ``, `"`, `,`.

This means the full Lisp identifier set is available in all surfaces: `list->string`, `ab-cd`, `zero?`, `box!`, `pair-of?` are all single tokens everywhere.

```
;; round
(list->string my-list)

;; curly
list->string(my-list);
```

```
;; spacy
list->string(my-list)
```

The mandatory parenthesization rule (§6.4) ensures infix expressions are unambiguous. Inside `(a op b)`, three space-separated tokens are parsed as infix. Outside of parenthesized infix, identifiers like `ab-cd` are never split — the lexer reads greedily until a delimiter.

```
;; curly
(ab-cd + ef-gh)          ;; infix: three tokens, + is the operator
ab-cd                    ;; one identifier
map(list->string xs);    ;; application: two arguments
```

No surface restricts the identifier character set. No underscore translation. No registry normalization across surfaces.

3.6 Argument separation

All three surfaces use **space** as the argument separator. There are no commas between arguments in any surface.

```
;; round
(f 3 4 5)

;; curly
f(3 4 5);

;; spacy
f(3 4 5)
```

Comma is reserved exclusively for pattern syntax: `,x` (bind), `,(x)` (catamorphism). This is universal across all three surfaces.

3.7 Surface equivalence

The following three definitions register the same content hash:

```
;; round
(define add (gamma ((,a ,b) (+ a b))))

;; curly
define add gamma { case (,a ,b): (a + b) };

;; spacy
define add: gamma:
    case (,a ,b): (a + b)
```

All three surfaces have identical capabilities. Any program written in one surface can be mechanically translated to either of the others. The ban on anonymous combiners as arguments is the constraint that makes this possible.

3.8 Comments

Round: `;` begins a line comment. `#;` is a datum comment — it comments out the next complete S-expression.

```
(+ 1 #;(ignored) 2)   ;; => 3
```

Curly: `//` begins a line comment. `;` is a statement separator and cannot double as a comment marker.

Spacy: `//` begins a line comment.

`#;` datum comments are available in all surfaces.

4. Gamma

`gamma` is the core foundation of Möbius. It takes a sequence of clauses and returns a combiner.

4.1 Basic form

Round:

```
(gamma (pattern₁ body₁)
       (pattern₂ body₂)
       ...
       (patternₙ bodyₙ))
```

Curly:

```
gamma {
    case pattern₁: body₁;
    case pattern₂: body₂;
    ...
    case patternₙ: bodyₙ;
}
```

Spacy:

```
gamma:
    case pattern₁: body₁
    case pattern₂: body₂
    ...
    case patternₙ: bodyₙ
```

The resulting combiner accepts a single tree argument, tries each pattern in order, and evaluates the body of the first matching clause.

4.2 Catamorphism

When a pattern uses `,(x)`, the enclosing gamma combiner is applied to the subtree recursively before binding. The body receives already-folded values. The programmer

never writes an explicit recursive call.

Round:

```
(define sum (gamma ((,head . ,(tail)) (+ head tail))
                   (#nil               0)))
(sum (list 1 2 3))
;; => 6
```

Curly:

```
define sum gamma {
    case (,head . ,(tail)): (head + tail);
    case #nil: 0;
};
sum(list(1 2 3));
;; => 6
```

Spacy:

```
define sum: gamma:
    case (,head . ,(tail)): (head + tail)
    case #nil: 0
sum(list(1 2 3))
;; => 6
```

Here `head` is bound to the raw car. `tail` is bound to the result of applying `sum` to the cdr. The fold is declared in the pattern, not the body.

Because recursion is limited to structural sub-parts of the matched value, catamorphic match over finite structures always terminates.

5. Patterns

Patterns describe the shape of a tree and how to bind parts of it. Pattern syntax is **identical across all three surfaces**.

5.1 Pattern forms

Literal. `42`, `"hello"`, `#true`, `#false`, `#nil` — matches that exact value.

Bind. `,x` — matches any value and binds it to `x`.

Recurse-and-bind. `,(x)` — matches any value, applies the enclosing gamma combiner to it recursively, and binds the result to `x`.

Wildcard. `,_` — matches any value, binds nothing.

Pair pattern. `(p₁ . p₂)` — matches a pair whose car matches `p₁` and cdr matches `p₂`.

List pattern. `(p₁ p₂ p₃)` — shorthand for nested pair patterns ending in `#nil`. `(,a ,b ,c)` expands to `(,a . (,b . (,c . #nil)))`.

Predicate guard. `(? pred ,x)` — applies `pred` to the value; if it returns a true value (anything other than `#false`), binds the value to `x`; if `#false`, the clause fails. The comma on the binding variable is mandatory.

5.2 No bare identifiers

Bare identifiers in patterns are forbidden. The registrar rejects any bare identifier in pattern position as an error.

```
;; WRONG — registration error
(gamma ((a b) 42))

;; CORRECT — bind two values
(gamma ((,a ,b) 42))

;; CORRECT — gamma literal values
(gamma ((0 1) 42))
```

This eliminates ambiguity between "match a resolved value" and "bind a new variable." There is no confusion.

5.3 The comma

The comma is the central syntactic device in Möbius patterns. It always means: computation has occurred before binding.

- `,x` — bind (extract the subtree).
- `,(x)` — recurse then bind (the enclosing gamma is applied to the subtree before binding).

Outside of patterns, comma has no syntactic role — it is **not** an argument separator in any surface.

5.4 Tail matching

Tail matching uses dot notation:

```
;; round
(gamma ((,a ,b . ,rest) (list a b rest)))

;; curly
gamma { case (,a ,b . ,rest): list(a b rest) }

;; spacy
gamma:
    case (,a ,b . ,rest): list(a b rest)
```

6. Evaluation

6.1 Application rule

An application expression is evaluated as follows:

1. Evaluate the combiner (the head).
2. Evaluate each argument left to right.
3. Construct the argument tree: `(cons v₁ (cons v₂ (... (cons vₙ #nil))))`.
4. Apply the combiner to that tree.

Round: `(f 3 4)` — evaluate `f`, evaluate `3`, evaluate `4`, construct `(3 . (4 . #nil))`, apply.

Curly: `f(3 4);` — same evaluation, different syntax.

Spacy: `f(3 4)` — same evaluation, different syntax.

A combiner that takes "two arguments" is a combiner whose gamma pattern destructures a two-element tree:

```
;; round
(define add (gamma ((,a ,b) (+ a b))))
(add 3 4)
;; argument tree is (3 . (4 . #nil)), pattern (,a ,b) matches
;; => 7
```

6.2 Tree in, tree out

Every combiner takes one argument (a tree) and returns one value (a tree). There is no `values` form and no `call-with-values`. If a procedure wants to return multiple things, it returns a tree containing them. The caller destructures the result with gamma.

```
;; round
(define f (gamma ((,x ,y) (cons (+ x y) (* x y)))))
(define g (gamma ((,sum . ,product) (list sum product))))
(g (f 3 4))
;; => (7 12)
```

6.3 Short-circuit forms

`if`, `and`, and `or` are the conditional foundations. All three have non-standard evaluation — they do not evaluate all their arguments.

`if` evaluates `test`; if the result is `#false`, evaluates `else`; for any other value, evaluates `then`.

```
;; round
(if test then else)

;; curly
```

```
if (test) { then } else { else };

;; spacy
if (test):
    then
else:
    else
```

Only `#false` triggers the else branch. `0`, `#nil`, `""`, and `#void` are all true.

`and` — short-circuit conjunction. Semantically equivalent to nested `if`:

```
(and a b c)
;; equivalent to:
(if a (if b c #false) #false)
```

Returns `#false` as soon as any argument is `#false`; otherwise returns the last value.

`or` — short-circuit disjunction. Semantically equivalent to nested `if` with temporary bindings to avoid double evaluation:

```
(or a b c)
```

Returns the first value that is not `#false`. If all are `#false`, returns `#false`.

6.4 Infix evaluation (curly and spacy)

In curly and spacy, infix expressions are fully parenthesized:

```
;; curly/spacy
(1 + 2)         ;; valid — reader produces (+ 1 2)
(1 + (2 * 3))   ;; valid — reader produces (+ 1 (* 2 3))
1 + 2            ;; READER ERROR — missing parentheses
1 + 2 * 3        ;; READER ERROR — no precedence
```

There is no operator precedence. Every infix expression must be explicitly parenthesized. The reader translates `(a op b)` to `(op a b)`, producing the same tree as round's prefix form.

An identifier like + in argument position (not infix) is just a combiner value:

```
;; curly
map(+ my-list);   ;; passes the combiner + as first arg to map
```

7. Define and Scope

7.1 Top-level define

`define` binds a name to a value.

Round: `(define name expression)`

Curly: `define name expression;`

Spacy: `define name: expression`

Top-level definitions are immutable and content-addressed. The expression may be a combiner (`gamma` or `lambda`), a constant (atom, pair, list), or any expression that evaluates to an immutable value. Boxes are forbidden in top-level definitions.

7.2 Content addressing

In the content-addressed representation, a defined combiner has no free variables. Every reference to another definition is resolved to a content hash.

```
;; round
(define x 10)
(define f (gamma ((,a) (+ a x))))
```

In the stored form, `f` does not contain a free reference to `x`. It contains the hash of `10`.

7.3 The dependency DAG

The dependency graph between top-level definition **units** is strictly a directed acyclic graph. The registrar detects mutually recursive definitions and bundles them into a single content-addressed unit. The hash covers the entire mutual group.

Round:

```
;; odd? and even? reference each other — the registrar bundles them
(define odd? (gamma ((0 #false)
                     (,n (even? (- n 1))))))
(define even? (gamma ((0 #true)
                      (,n (odd? (- n 1))))))
```

Curly:

```
define odd? gamma {
    case 0: #false;
    case ,n: even?((n - 1));
};
define even? gamma {
    case 0: #true;
    case ,n: odd?((n - 1));
};
```

Spacy:

```
define odd?: gamma:
    case 0: #false
    case ,n: even?((n - 1))
define even?: gamma:
    case 0: #true
    case ,n: odd?((n - 1))
```

Stale references. If the programmer later updates `odd?` without updating `even?`, the result is:

`odd?₁ → even?₀ → odd?₀`

This is a valid DAG — no structural problem. But it is a semantic bug: `even?₀` still calls the old `odd?₀`, not the new `odd?₁`. The content store faithfully preserves the stale reference. Tooling should warn about broken mutual groups, but the language does not forbid them.

7.4 Nested define

Within a combiner body, `define` creates local bindings. Nested definitions are mutually visible (hoisted), enabling local mutual recursion.

7.5 No let, no let*

Möbius has no `let`, `let*`, or `letrec` as separate binding forms. All local binding is done through nested `define` within a `begin` block (round), brace block (curly), or indented block (spacy). Since nested defines are mutually visible, this subsumes `letrec`.

7.6 Sequencing

Round: `(begin e₁ e₂ ... eₙ)` — evaluates each expression in order, returns the value of `eₙ`.

Curly: `{ e₁; e₂; ... eₙ }` — braces and semicolons. Last expression is the return value.

Spacy: Newlines under the same indentation level. Last expression is the return value.

`begin` is a universal concept — only its surface syntax varies.

8. Lambda

`lambda` is a foundation that constructs a single-clause `gamma` with all-bind patterns. Semantically equivalent to `gamma`: `(lambda (a b) body)` = `(gamma ((,a ,b) body))`. The compiler may exploit this equivalence.

Equivalence:

```
;; (lambda (a b c) body) is equivalent to:
;; (gamma ((,a ,b ,c) body))
```

Lambda parameters are always bare names (no comma) — all parameters are binds by definition.

Round:

```
(define add (lambda (a b) (+ a b)))
```

Curly:

```
define add lambda (a b) { (a + b) };
```

Spacy:

```
define add: lambda (a b): (a + b)
```

Lambda must always be explicit. There is no `(define (f x) body)` shorthand. If you want a combiner, you write `gamma` or `lambda`.

9. Capsules

A capsule type is defined by a 128-bit type identifier. The foundation `encapsulation-type` takes this identifier and returns a tree of three combiners.

Round:

```
(define my-type (encapsulation-type 0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8))
(define make-my (car my-type))
(define my? (car (cdr my-type)))
(define unwrap-my (car (cdr (cdr my-type))))
```

Curly:

```
define my-type encapsulation-type(0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8);
define make-my car(my-type);
define my? car(cdr(my-type));
define unwrap-my car(cdr(cdr(my-type)));
```

Spacy:

```
define my-type: encapsulation-type(0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8)
define make-my: car(my-type)
define my?: car(cdr(my-type))
define unwrap-my: car(cdr(cdr(my-type)))
```

Constructor (`car`): wraps any tree in an opaque capsule tagged with this type ID.

Predicate (`car` of `cdr`): returns `#true` if a value is a capsule with this type ID, `#false` otherwise.

Accessor (`car` of `cdr` of `cdr`): unwraps the capsule, returning the inner tree. Fails if the value does not have this type ID.

9.1 Non-generative types

Capsule types are **non-generative**: the same 128-bit identifier anywhere defines the same type. Two modules using the same ID have compatible types. Two modules using different IDs have incompatible types, even if structurally identical.

The 128-bit identifier is an atom — it lives in the content store like any other data. There is no runtime generation of fresh types.

Collision. Because type identifiers are chosen by the programmer, two unrelated capsule types may accidentally share the same ID. This is a bug, not a feature. Linting tools should detect duplicate type IDs across a codebase. The spec recommends deriving type IDs deterministically (e.g., from a hash of the defining module's content and a local name) rather than choosing them by hand.

9.2 Capsules and predicates

A capsule's predicate is its type. There is no separate type declaration language. A type is a question: does this value satisfy this predicate?

Predicates compose freely. A value may satisfy multiple predicates without any hierarchical relationship. There is no inheritance, no class hierarchy. Classification is external and open.

9.3 Capsules and mutability

A capsule wraps any value — including boxes. If a capsule contains a box (or a tree containing boxes), the capsule's contents are mutable through `box!`. The capsule itself is opaque and cannot be swapped out, but boxes inside it can be mutated.

A capsule wrapping only atoms and pairs is immutable. A capsule wrapping a box is a stateful object. The distinction is determined by what the programmer puts inside.

9.4 Decapsulation

Decapsulation is always explicit. Code must use the accessor to unwrap a capsule before matching on its contents. Pattern-level capsule destructuring is not supported — use a predicate guard to dispatch on type, then call the accessor in the body.

```
;; round
(gamma (((? my? ,x) (do-something (unwrap-my x)))
        (,other     (do-other other))))

;; curly
gamma {
    case (? my? ,x): do-something(unwrap-my(x));
    case ,other: do-other(other);
}

;; spacy
gamma:
    case (? my? ,x): do-something(unwrap-my(x))
    case ,other: do-other(other)
```

10. Boxes

A box is a mutable indirection cell. Boxes are the only mutable values in Möbius.

10.1 Foundations

- `(box v)` — create a new box containing the value `v`.
- `(unbox b)` — return the current contents of box `b`.
- `(box! b v)` — replace the contents of box `b` with `v`. Returns `#void`.

10.2 Restrictions

Boxes are **forbidden in top-level definitions**. A top-level `define` must bind an immutable, content-addressable value. Boxes exist only at runtime, within nested scope:

Round:

```
(define make-counter
  (lambda ()
    (begin
      (define state (box 0))
      (gamma (("get"  (unbox state))
              ("inc"  (begin (box! state (+ (unbox state) 1))
                             (unbox state))))))))
```

Curly:

```
define make-counter lambda () {
    define state box(0);
    gamma {
        case "get": unbox(state);
        case "inc": {
            box!(state (unbox(state) + 1));
            unbox(state)
        };
    }
};
```

Spacy:

```
define make-counter: lambda ():
    define state: box(0)
    gamma:
        case "get": unbox(state)
        case "inc":
            box!(state (unbox(state) + 1))
            unbox(state)
```

10.3 Box identity

Each `box` call creates a fresh, distinct cell. `(eq? (box 0) (box 0))` is `#false`.

`equal?` on boxes compares their current contents recursively — it is a snapshot comparison.

10.4 Self-referential boxes

A box may contain itself:

```
(define b (box #nil))
(box! b b)
(eq? (unbox b) b)   ;; => #true
```

This is permitted. `equal?` on cyclic box structures is undefined behavior — implementations may diverge, detect the cycle, or signal an error.

11. Predicates and Typing

Möbius's type discipline is based on predicate calculus rather than object-oriented classification.

Where OOP asks "what IS this value" (identity), Möbius asks "what is TRUE of this value" (predication). The distinction matters:

- A value can satisfy any number of predicates.
- Predicates are ordinary combiners.
- No type must be anticipated at definition time.
- Adding new predicates never requires modifying existing code.

Type inference in Möbius means: given the predicates that hold of a combiner's input, which predicates can be proved to hold of its output?

The formal inference mechanism is to be specified in a future revision.

12. Continuations and Control Flow

Möbius provides three continuation foundations and one well-known continuation binding for non-local control flow. Because content-addressed definitions have no free variables, these foundations operate entirely at runtime on the continuation — the chain of pending computation — rather than on lexical environments.

12.1 call/cc

`call/cc` takes a single combiner as argument and calls it with a first-class continuation object representing the current point of execution. This continuation is unlimited — it captures the entire future of the computation. The continuation is a value like any other and may be stored in a tree, returned, or passed to other combiners. It remains valid indefinitely, including after the dynamic extent of the `call/cc` has exited.

If the continuation is never applied, it has no effect. The combiner may return normally, in which case the result of `call/cc` is whatever the combiner returns.

12.2 continuation-apply

`continuation-apply` takes a continuation object and a value, and delivers that value to the captured continuation. This abandons the current computation entirely — control jumps to the point captured by `call/cc`, and the delivered value becomes the result of that original `call/cc` expression.

If the continuation crosses guard boundaries, the corresponding guard clauses are invoked during the pass. Applying a continuation that was captured in a dynamic extent that has already exited is permitted. Applying the same continuation multiple times is permitted.

12.3 guard

`guard` installs entry and exit gamma clauses on a continuation boundary and executes a thunk within that boundary. It merges the roles of Scheme's `guard` (exception handling) and `dynamic-wind` (entry/exit behavior) into a single pattern-based mechanism.

Round:

```
(guard
  (entry (pattern₁ handler₁)
         (pattern₂ handler₂)
         ...)
  thunk
  (exit (pattern₁ handler₁)
        (pattern₂ handler₂)
        ...))
```

Curly:

```
guard {
    entry:
        case pattern₁: handler₁;
        case pattern₂: handler₂;
    body: thunk;
    exit:
        case pattern₁: handler₁;
        case pattern₂: handler₂;

};
```

Spacy:

```
guard:
    entry:
        case pattern₁: handler₁
        case pattern₂: handler₂
    body: thunk
    exit:
```

```
case pattern₁: handler₁
case pattern₂: handler₂
```

Entry clauses. When an abnormal pass (a value delivered via `continuation-apply`) crosses into this guard boundary from outside, the passed value is matched against the entry clauses in order. If a clause matches, its body executes to handle the pass. If no clause matches, the pass propagates automatically.

Thunk. A zero-argument combiner that executes as the guarded body. The thunk runs within the protection of the guard boundary.

Exit clauses. When an abnormal pass crosses out of this guard boundary to outside, the passed value is matched against the exit clauses. This is the cleanup mechanism — exit clauses run when control leaves the guarded region, whether normally or via continuation.

Guards are the sole mechanism for intercepting non-local control flow. They subsume exception handling (entry clauses matching error values), cleanup (exit clauses performing side effects then propagating), and dynamic-wind entry/exit behavior (entry and exit clauses on the same boundary).

The key edge case: if a guard clause itself signals an abnormal exit, that exit propagates outward past the current guard — a guard does not intercept its own errors.

12.4 The root continuation (continuation-exit)

`continuation-exit` is a well-known binding — not a separate foundation, but the root continuation that exists when a program starts. It represents "terminate the process."

Delivering a value to `continuation-exit` terminates the program. The delivered value must be an integer 0–255 (POSIX exit code). Delivering a non-integer or out-of-range value is an error.

```
;; round
(continuation-apply continuation-exit 0)   ;; exit successfully

;; curly
continuation-apply(continuation-exit 0);

;; spacy
continuation-apply(continuation-exit 0)
```

Guards installed between the current point and the root continuation are traversed on exit, so cleanup code runs.

If the program's main expression returns normally (without explicitly calling `continuation-exit`), the runtime delivers `0` to the root continuation — successful termination.

13. Reader Syntax

The reader converts a character stream into trees. It is not part of the language semantics — it is one possible surface syntax. This section specifies the round (S-expression) reader. The curly and spacy readers (§3.3, §3.4) produce identical trees through different surface conventions.

13.1 Atoms

Integers. A sequence of digits, optionally preceded by `-` or `+`. Examples: `42`, `-7`, `0`, `+3`.

Floats. Digits with a decimal point, optionally preceded by a sign, optionally followed by an exponent. Examples: `3.14`, `-0.5`, `1e10`, `2.5e-3`.

Characters. `#\` followed by a single character or a character name. Examples: `#\a`, `#\Z`, `#\space`, `#\newline`, `#\tab`. The reader recognizes at least `space`, `newline`, `tab`, and `return`.

Strings. Delimited by double quotes. Escape sequences: `\\`, `\"`, `\n`, `\t`, `\r`.

Type identifiers. A 128-bit integer in hexadecimal with `0x` prefix. Example: `0x9f3a7b2c4d5e6f708192a3b4c5d6e7f8`.

13.2 Hash-identifiers

A `#` followed by one or more alphanumeric characters or hyphens. Hash-identifiers are identifiers — resolved at registration time like symbols.

Four are pre-defined:

- `#true` — the truthy value.
- `#false` — the sole false value.
- `#nil` — the empty list.
- `#void` — the void singleton, returned by side-effecting operations.

All others are regular identifiers. By convention, a hash-identifier names a singleton:

```
(define #eof-type (encapsulation-type 0x00000000000000000000000000000001))
(define #eof ((car #eof-type) #nil))
```

13.3 Symbols (surface syntax only)

An identifier starts with an alphabetical character or one of `+ - * / < > = ? ! _` and continues until whitespace or a structural delimiter: `( ) { } ; : ' `` " ,`. This rule is the same across all three surfaces (§3.5).

Examples: `foo`, `car`, `+`, `list->string`, `point?`, `encapsulation-type`, `ab-cd`, `box!`.

Symbols are surface syntax only. At registration time, they are resolved to content hashes and disappear. They do not exist as runtime values.

13.4 Parentheses and pairs

`(a b c)` reads as `(cons a (cons b (cons c #nil)))` — a proper list.

`(a . b)` reads as `(cons a b)` — a pair.

`(a b . c)` reads as `(cons a (cons b c))` — an improper list.

Dot notation in expression position is reader shorthand for `cons`, not a distinct expression form.

13.5 Comments

`;` begins a line comment (round surface). `//` begins a line comment (curly and spacy surfaces).

`#;` is a datum comment in all surfaces — it comments out the next complete expression:

```
(+ 1 #;(ignored) 2)   ;; => 3
```

14. Foundations and Base Library

The vocabulary of Möbius is organized by a single criterion: **can it be written in Möbius?**

Foundations are forms and combiners that require the evaluator or runtime. They cannot be expressed as Möbius programs. Some foundations have equivalent expansions in terms of other foundations — `lambda` in terms of `gamma`, `and` in terms of `if`. These equivalences are semantic facts the compiler may exploit, not a hierarchy of dependence. The language *chooses* to present all of them as building blocks.

Base library combiners are Möbius programs. They have content hashes and live in the store. They are the first programs written in the language, shipped alongside it. Any programmer could write them.

14.1 Foundations

Core forms (special evaluation rules):

Foundation	Role
`gamma`	Tree destructuring, binding, catamorphism, combiner construction
`if`	Conditional — only `#false` triggers the else branch
`and`	Short-circuit conjunction. Equivalent to nested `if`: `(and a b c)` = `(if a (if b c #false) #false)`

Foundation	Role
`or`	Short-circuit disjunction. Returns first non-`#false` value. Equivalent to nested `if` with temporary bindings.
`lambda`	Single-clause `gamma` with all-bind patterns. `(lambda (a b) body)` = `(gamma ((,a ,b) body))`
`begin`	Sequencing (round surface; braces in curly, indentation in spacy)
`define`	Bind a name (top-level: content hash; nested: local binding)
`guard`	Install entry/exit gamma clauses on a continuation boundary

Data construction and access:

Foundation	Role
`cons`	Construct a pair
`car`	First element of a pair
`cdr`	Second element of a pair
`encapsulation-type`	Create a capsule type from a 128-bit ID (constructor, predicate, accessor)
`box`	Create a mutable box
`unbox`	Read box contents
`box!`	Mutate box contents

Continuations:

Foundation	Role
`call/cc`	Reify the current continuation
`continuation-apply`	Deliver a value to a captured continuation

Type predicates:

Foundation	Role
`integer?`	Test if value is an integer
`float?`	Test if value is a float
`char?`	Test if value is a character
`string?`	Test if value is a string
`pair?`	Test if value is a pair

Foundation	Role
`box?`	Test if value is a box
`combiner?`	Test if value is a combiner
`continuation?`	Test if value is a continuation

Comparison and arithmetic:

Foundation	Role
`eq?`	Identity comparison
`+`, `-`, `*`, `/`	Arithmetic
`<`, `>`, `=`	Comparison

I/O:

Foundation	Role
`display`	Write a value to standard output. Returns `#void`.

Well-known bindings (values bound in the initial environment):

Binding	Role
`continuation-exit`	The root continuation — delivers an exit code 0–255 to terminate
`#true`, `#false`, `#nil`, `#void`	Pre-defined constants

Foundation count: 8 core forms + 7 data + 2 continuations + 8 type predicates + 8 arithmetic/comparison + 1 I/O = **34 foundations**.

14.2 Base library

Möbius programs shipped with the system. They have content hashes and live in the content store.

Name	Definition	Role
`list`	`(gamma (,args args))`	Identity — returns the argument tree. `(list 1 2 3)` constructs `(1 . (2 . (3 . #nil)))` via the application rule and returns it unchanged.

Name	Definition	Role
`not`	`(gamma ((#false #true) (,_ #false)))`	Boolean negation
`equal?`	Recursive structural comparison using `gamma`, `pair?`, `eq?`, `car`, `cdr`, `unbox`, `box?`. Pairs compared element-wise. Capsules opaque (only `eq?`). Boxes compared by current contents.	Deep equality
`continuation-extend`	Built from `call/cc` and `continuation-apply`. Takes a continuation and a combiner, returns a new continuation such that delivering `v` applies `f` to `v` first, then delivers the result to `k`.	Continuation composition

Note on `eq?` vs `equal?`: `eq?` tests identity — whether two values are the same object. For atoms, `eq?` compares values. For pairs, `eq?` compares identity. For capsules, `eq?` compares identity. For boxes, `eq?` compares cell identity. `equal?` recurses through pairs and box contents. Capsules are opaque to `equal?` — two capsules are `equal?` only if they are `eq?`.

15. Grammar

15.1 Round surface grammar

```
program     ::= expression*

expression  ::= atom
              | identifier
              | (gamma clause+)
              | (if expression expression expression)
              | (begin expression+)
              | (define identifier expression)
              | (guard guard-body)
              | (lambda formals expression)        ;; foundation
              | (and expression+)                  ;; foundation
              | (or expression+)                   ;; foundation
              | (expression expression*)

clause      ::= (pattern expression)
```

```
guard-body  ::= (entry clause*) (exit clause*) expression

formals     ::= (identifier*)

pattern     ::= atom                       ;; literal gamma
              | ,identifier                ;; bind
              | ,(identifier)              ;; recurse-and-bind
              | ,_                         ;; wildcard
              | (? identifier ,identifier) ;; predicate guard
              | (pattern . pattern)        ;; pair
              | (pattern*)                 ;; list (shorthand for nested pairs)

atom        ::= integer | float | character | string | type-id

integer     ::= [+-]? digit+
float       ::= [+-]? digit+ '.' digit* ([eE] [+-]? digit+)?
character   ::= '#\' (character-name | any-character)
string      ::= '"' string-char* '"'
type-id     ::= '0x' hex-digit+

identifier  ::= symbol | hash-id
symbol      ::= id-start id-continue*
id-start    ::= alpha | '+' | '-' | '*' | '/' | '<' | '>' | '=' | '?' | '!' | '_'
id-continue ::= (any character except whitespace and delimiters: ( ) { } ; : ' ` " ,)
hash-id     ::= '#' (alpha | digit | '-')+
```

15.2 Curly surface grammar

```
program     ::= statement*

statement   ::= define-stmt | expression ';'

define-stmt ::= 'define' identifier expression ';'
              | 'define' identifier gamma-expr ';'
              | 'define' identifier lambda-expr ';'

expression  ::= atom
              | identifier
              | gamma-expr
              | lambda-expr
              | guard-expr
              | identifier '(' args ')'           ;; application
              | '(' expression op expression ')'  ;; infix
              | 'if' '(' expression ')' block
                ('else' block)?
```

```
              | '{' statement* expression '}'      ;; begin block

gamma-expr  ::= 'gamma' '{' case-clause+ '}'

case-clause ::= 'case' pattern ':' expression ';'
              | 'case' pattern ':' block ';'

lambda-expr ::= 'lambda' '(' identifier* ')' block

guard-expr  ::= 'guard' '{'
                  'entry' ':' case-clause*
                  'exit' ':' case-clause*
                  'body' ':' expression ';'
                '}'

block       ::= '{' statement* expression '}'

args        ::= expression*

op          ::= '+' | '-' | '*' | '/' | '<' | '>' | '='
              | identifier                        ;; any combiner as infix

pattern     ::= (same as round — §15.1)
```

15.3 Spacy surface grammar

```
program     ::= line*

line        ::= define-line | expression

define-line ::= 'define' identifier ':' expression
              | 'define' identifier ':' gamma-block
              | 'define' identifier ':' lambda-line

expression  ::= atom
              | identifier
              | identifier '(' args ')'           ;; application
              | '(' expression op expression ')'  ;; infix
              | if-expr

gamma-block ::= 'gamma' ':' INDENT case-line+ DEDENT

case-line   ::= 'case' pattern ':' expression
              | 'case' pattern ':' INDENT line+ DEDENT

lambda-line ::= 'lambda' '(' identifier* ')' ':' expression
```

```
guard-block ::= 'guard' ':' INDENT
                  'entry' ':' INDENT case-line* DEDENT
                  'exit' ':' INDENT case-line* DEDENT
                  'body' ':' expression
                DEDENT

if-expr     ::= 'if' '(' expression ')' ':' INDENT
                  line+ DEDENT
                'else' ':' INDENT line+ DEDENT

args        ::= expression*

op          ::= (same as curly — §15.2)

pattern     ::= (same as round — §15.1)

INDENT      ::= (increase in indentation level)
DEDENT      ::= (decrease in indentation level)
```

Note: anonymous lambda/gamma as arguments is not supported in any surface. Use a named `define` instead.

16. Open Questions

The following are identified for future revisions:

1. **Predicate inference.** The compiler infers predicates from known properties of foundations — `car` requires `pair?`, `+` requires `integer?` or `float?`. Gamma clause structure propagates these predicates. The programmer may add assert hints. The formal specification of the inference mechanism, the set of foundation predicate signatures, and the interaction between capsule-level inference and tree-level predicates remain to be detailed.

2. **Effects.** Effects in Möbius are named patterns over existing continuation foundations, not new foundations. `raise` (one-shot, no resumption), `raise-continuable` (one-shot resumption), and coroutines (multi-shot, sequential resumption) are defined in terms of `guard`, `call/cc`, `continuation-apply`, and `gamma`. The compiler recognizes these named patterns and optimizes accordingly: `raise` compiles to a jump, `raise-continuable` to a call, coroutines to stack switching. The formal definitions remain to be specified.

3. **eval semantics.** Without symbols as values and without quote, `eval` in the traditional Lisp sense is not possible. What remains is hash lookup: given a content hash, retrieve and execute the corresponding combiner from the content store. Open questions: Is this a foundation, or is it implicit in application? What happens when a hash is not in the content store?

4. **Error model.** What happens when a gamma fails (no clause matches)? When `car` is applied to an atom? When division by zero occurs? The spec needs to define: what an error value is (likely a capsule), how errors are raised (likely via `continuation-apply` to the nearest guard), and how they interact with `guard`. This is critical for implementers.

5. **Registry representation.** How is the registry represented and stored? How are registries versioned — append-only logs, content-addressed snapshots, or both? How do multiple registries compose? How are multilingual name bindings managed? Registry modes: local-only, federated, centralized.

6. **Concurrency.** Möbius's tree-in-tree-out model and immutable content store are natural fits for concurrent and distributed computation. CSP-style channels (read-channel, write-channel, select) are a candidate model. The interaction between concurrency, boxes, and continuations needs careful specification.

7. **Cycle detection for `equal?` on boxes.** Self-referential boxes are permitted. `equal?` on cyclic box structures is currently undefined behavior. Should implementations be required to detect cycles?

8. **I/O model.** `display` is a foundation but I/O is otherwise unspecified. How do file handles, network sockets, and other resources interact with the content-addressed model? Are they capsules wrapping OS handles?

17. Interpreter Milestones

The möbius-zero interpreter should be built in phases:

Phase 1 — Minimal gamma. Round reader (S-expressions). Values: atoms (including 128-bit type identifiers), pairs, empty list, booleans, `#void`, combiners. `gamma` with: literal, `,x` bind, pair pattern, list pattern, `,_` wildcard, predicate guard `(? pred ,x)`. Foundations: `define`, `if`, `and`, `or`, `lambda`, `begin`, `cons`, `car`, `cdr`, `eq?`, arithmetic, comparison, type predicates (`integer?`, `float?`, `pair?`, `string?`, `char?`), `display`. Base library: `list`, `not`. Application rule per §6.1. Left-to-right evaluation. No bare identifiers in patterns (rejected at registration). Registrar bundles mutually recursive definitions.

Phase 2 — Recursion. Catamorphic `,(x)` in gamma patterns. Nested `define` with mutual visibility within a body.

Phase 3 — Abstraction. `encapsulation-type` with 128-bit identifiers. Capsule constructor, predicate, accessor. Non-generative types. Explicit decapsulation before matching.

Phase 4 — Mutation. `box`, `unbox`, `box!`. Forbidden at top level. Runtime-only. Type predicates: `box?`, `combiner?`, `continuation?`. Base library: `equal?`.

Phase 5 — Control. `call/cc`, `continuation-apply`, `guard` with entry/exit clauses. `continuation-exit` as root continuation. Base library: `continuation-extend`.

Phase 6 — Surfaces. Curly reader. Spacy reader. Surface declaration (`#lang`). All three readers produce identical trees. Verify hash equivalence across surfaces.

This document records the design of Möbius as understood in February 2026. It is a working specification, not a final standard. The "0" in R0RM reflects this: it is the revision before revision, the seed before the tree.

Annexes

Non-normative — possible future additions and compiler notes

Annex A. Ellipsis (Possible Future Addition)

The ellipsis `...` in a pattern would mean "zero or more repetitions of the preceding sub-pattern." Each `...` would add one depth level to the bindings it contains. In the body, `...` would expand those bindings at the corresponding depth. This section sketches the design; it is not part of the current language.

A.1 Basic iteration — depth 1

```
;; collect all elements
((gamma ((,a ...) (,a ...)))
 (list 1 2 3))
;; => (1 2 3)

;; transform each element
((gamma ((,a ...) ((* 2 a) ...)))
 (list 1 2 3))
;; => (2 4 6)
```

`a` is bound at depth 1. In the body, `(* 2 a) ...` iterates over `a`.

A.2 Parallel bindings

```
((gamma (((,a ,b) ...) ((+ a b) ...)))
 (list (list 1 10) (list 2 20) (list 3 30)))
;; => (11 22 33)
```

Mismatched lengths are an error.

A.3 Nested ellipsis — depth 2

```
;; flatten a list of lists
((gamma (((,a ...) ...) (,a ... ...)))
```

```
 (list (list 1 2) (list 3 4) (list 5 6)))
;; => (1 2 3 4 5 6)

;; double every element, preserving structure
((gamma (((,a ...) ...) (((* 2 a) ...) ...)))
 (list (list 1 2) (list 3 4)))
;; => ((2 4) (6 8))

;; double every element, then flatten
((gamma (((,a ...) ...) ((* 2 a) ... ...)))
 (list (list 1 2) (list 3 4)))
;; => (2 4 6 8)
```

A.4 Depth rules

- Each `...` in a pattern adds one depth level.
- In the body, each `...` consumes one depth level.
- A depth-0 binding under `...` is repeated at each iteration.
- Multiple bindings at the same depth must have the same length.
- Depth mismatch is an error.

Restrictions: `...` only at the tail of a list pattern. `(,a ... ,b)` is not allowed. `(,a ... . ,rest)` is not supported.

Interaction with catamorphism: Ellipsis and `,(x)` are orthogonal. They can appear in the same gamma.

Annex B. Predicate-Driven Optimization

Predicate information drives compiler optimization:

Representation narrowing. A value satisfying `uint4?` needs only 4 bits.

Pointer tagging. Known predicate constraints free up tag bits.

Dead code elimination. Gamma clauses ruled out by predicates are unreachable.

Check elimination. Runtime checks implied by known predicates are redundant.

Specialization. Known predicates enable specialized code paths.

Annex C. Representation Inference

The only compositional data constructor is `cons`. The compiler infers optimal machine representations from predicate knowledge:

Byte vectors. A list of `uint8?` values with known length → contiguous bytes.

Arrays. Homogeneous known-length list → flat array with O(1) indexed access.

Structs. Fixed-position pair structure with known types → struct with fixed offsets.

Hash tables. Association list consistently accessed by key → amortized O(1) lookup.

Capsules are the natural boundary where representation choices stabilize. The capsule's opacity guarantees no external code depends on internal layout, freeing the compiler to change representations without breakage.

Annex D. Knowledge Economy

Reference: Aaron Hsu, "Programming Obesity: A Code Health Epidemic" (FnConf 2019)

Hsu proposes three axes for evaluating programming systems: simplicity (structural complexity), speed (execution performance), and economy (how much knowledge scales to a broad set of problems).

Economy is measured as the ratio of domain-specific names to total names in a program. Fewer unique concepts → less code → faster execution → more knowledge reuse. These compound rather than trade off.

Möbius is designed for high economy: one data type (tree), one mechanism (`gamma`), ~34 foundations. No frameworks, no DSLs, no separate type language. The programmer's attention is spent on the domain problem, not on systems rumination.

Annex E. Abacus — An Arithmetic Evaluator

This annex demonstrates catamorphic match on a concrete problem: evaluating arithmetic expression trees. The entire evaluator is one `gamma` with no explicit recursion.

E.1 Representation

Expression trees use string tags as node labels:

```
;; round
(list "add" 1 (list "mul" 2 3))          ;; 1 + (2 * 3) = 7
(list "sub" (list "add" 10 5) 3)         ;; (10 + 5) - 3 = 12
(list "mul" (list "add" 1 2) (list "add" 3 4)) ;; (1 + 2) * (3 + 4) = 21
```

A number is a leaf. A tagged list `("op" left right)` is a branch. This is a tree — `cons` all the way down.

E.2 The evaluator

Round:

```
(define abacus
  (gamma ((? integer? ,n) n)
         (("add" ,(left) ,(right)) (+ left right))
         (("sub" ,(left) ,(right)) (- left right))
         (("mul" ,(left) ,(right)) (* left right))
         (("div" ,(left) ,(right)) (/ left right))))
```

```
(abacus (list "add" 1 (list "mul" 2 3)))
;; => 7

(abacus (list "mul" (list "add" 1 2) (list "add" 3 4)))
;; => 21
```

Curly:

```
define abacus gamma {
    case (? integer? ,n): n;
    case ("add" ,(left) ,(right)): (left + right);
    case ("sub" ,(left) ,(right)): (left - right);
    case ("mul" ,(left) ,(right)): (left * right);
    case ("div" ,(left) ,(right)): (left / right);
};

abacus(list("add" 1 list("mul" 2 3)));
;; => 7
```

Spacy:

```
define abacus: gamma:
    case (? integer? ,n): n
    case ("add" ,(left) ,(right)): (left + right)
    case ("sub" ,(left) ,(right)): (left - right)
    case ("mul" ,(left) ,(right)): (left * right)
    case ("div" ,(left) ,(right)): (left / right)

abacus(list("add" 1 list("mul" 2 3)))
;; => 7
```

E.3 How it works

The key is `,(left)` and `,(right)`. These are catamorphic binds — before the body runs, `abacus` is applied recursively to each subtree.

Trace of `(abacus (list "add" 1 (list "mul" 2 3)))`:

1. Input matches `("add" ,(left) ,(right))`.
2. `left`: apply `abacus` to `1`. Matches `(? integer? ,n)`. Returns `1`.
3. `right`: apply `abacus` to `("mul" 2 3)`. Matches `("mul" ,(left) ,(right))`.
   - `left`: apply `abacus` to `2`. Returns `2`.
   - `right`: apply `abacus` to `3`. Returns `3`.
   - Body: `(* 2 3)` → `6`.
4. Body: `(+ 1 6)` → `7`.

The programmer writes no recursive calls. The fold is declared in the pattern. The body only sees already-computed values.

E.4 Extending the evaluator

Adding negation — a unary operation:

Round:

```
(define abacus
  (gamma ((? integer? ,n) n)
         (("add" ,(left) ,(right)) (+ left right))
         (("sub" ,(left) ,(right)) (- left right))
         (("mul" ,(left) ,(right)) (* left right))
         (("div" ,(left) ,(right)) (/ left right))
         (("neg" ,(inner))         (- 0 inner))))

(abacus (list "neg" (list "add" 1 2)))
;; => -3
```

Each new operation is one clause. The catamorphism handles the recursion automatically — `,(inner)` evaluates the subtree before the body sees it.

E.5 What this demonstrates

- **One mechanism.** `gamma` handles dispatch, destructuring, recursion, and result construction. No visitor pattern, no interpreter loop, no recursive function definition.
- **The pattern is the program.** The shape of the clause *is* the specification of what the evaluator does. Reading the pattern tells you the input structure; reading the body tells you the output.
- **Catamorphism terminates.** Because `,(x)` only recurses into strict sub-parts of the matched value, the evaluator terminates on any finite expression tree. This is a structural guarantee, not a proof obligation.
- **Economy.** The evaluator uses 6 names from Möbius (`gamma`, `integer?`, `+`, `-`, `*`, `/`) and 5 from the domain (`"add"`, `"sub"`, `"mul"`, `"div"`, `"neg"`). Half the program is the problem; half is the tool.